

Project Overview: kfz-kosten

This project appears to be a command-line application written in Go for managing vehicle (KFZ) costs, including fuel-ups (**Tanken**) and other expenses (**Kosten**). It provides functionalities for data entry, data persistence, and generating reports, including an Excel export.

Core Components and Functionalities:

1. `go.mod`:

- Module Name: `kfz-kosten`
- Go Version: `1.23`
- Key Dependencies:
 - `github.com/jeschu/go-config`: Likely used for configuration management, specifically for loading and saving KFZ data (e.g., `kfzs.yaml`).
 - `github.com/xuri/excelize/v2`: Used for generating Excel reports.
 - `golang.org/x/term`: Used for terminal interaction, specifically for raw input reading in `input.go`.
 - `golang.org/x/text`: Used for text formatting, particularly for German language formatting in `model/printer.go`.

2. `main.go`:

- Main Application Entry Point: Orchestrates the user interaction and main program flow.
- Actions: Presents a menu-driven interface to the user with the following options:
 - **Tanken erfassen** (Record Fuel-up)
 - **Kosten erfassen** (Record Expense)
 - **Excel Export**
 - **Übersicht anzeigen** (Show Summary - default action on Enter)
- Data Persistence: Loads KFZ data at startup and saves it after **Tanken** or **Kosten** entries.
- Flow:
 - Loads existing KFZ data.
 - Enters a loop to present action choices.
 - Calls respective functions (`tanken`, `kosten`, `excel`) based on user selection.
 - Exits the loop to print a summary (`kfz.PrintStats()`).

3. `Makefile`:

- `show-config`: Displays the content of `~/Library/Application Support/kfzs.yaml`.
- `edit-config`: Opens `~/Library/Application Support/kfzs.yaml` in a code editor (assumes `code`

command is available).

- `clean`: Removes generated files (`version.go`, `build` directory).
- `generate`: Runs `go generate`.
- `build`: Creates a `build` directory and compiles the Go application into it.
- `install`: Cleans, builds, and then installs the Go application.
- `dependencies-upgrade`: Updates Go dependencies and tidies the `go.mod` file.

4. `input/input.go`:

- Provides utility functions for reading various types of user input from the command line.
- `ReadString`: Reads a generic string.
- `ReadInt`: Reads an integer.
- `ReadFloat64`: Reads a float64.
- `ReadSelectionMapped` / `ReadSelectionMapper` / `ReadSelection`: Handles reading user selections from a predefined set of options, including mapping input to custom types.
- `ReadDateInPast`: Reads a date from the user, defaulting to today and allowing partial date input (e.g., "15" for 15th of current month/year).
- `NormalizeGermanNumber`: Converts German number formats (e.g., "1.234,56") to standard Go float/int parsable formats (e.g., "1234.56").
- Uses `golang.org/x/term` for raw terminal input, enabling single-character input without requiring Enter.

5. `lang/lang.go`:

- Contains utility functions for language-specific formatting.
- `FixedString`: Pads or truncates a string to a fixed size, adding an ellipsis if truncated.
- `FormatDuration`: Formats a `time.Duration` into a human-readable string (e.g., "1 Jahr, 2 Tage, 3 Stunden").

6. `model/model.go`:

- Defines the core data structures for the application.
- `Kfzs`: A map of `string` to `Kfz`, representing multiple vehicles.
- `Kfz`: Represents a single vehicle with `Name`, `Kennzeichen` (license plate), and slices of `Kosten` and `Tanken` entries.
- `Kosten`: Represents an expense entry with `Datum` (date), `Km` (odometer reading), `Kategorie`, `AbschreibungKm` (depreciation by km), `AbschreibungZeit` (depreciation by time), `AbschreibungFa` (tax office relevance), `Kosten` (cost), and `Notiz` (note).
- `Tanken`: Represents a fuel-up entry with `Datum`, `Art` (type: Full, Partial, First), `Km`, `Liter`, `Kosten`, and `Sorte` (fuel type).
- `TankArt`: Enum for fuel-up types (`Erst`, `Teil`, `Voll`).
- `Finanzamt`: Enum for tax office relevance (`Nein`, `Ja`, `Abschreibung`).
- Business Logic Methods:

- `MaxKm()`: Returns the maximum odometer reading and its corresponding date across all `Kosten` and `Tanken` entries.
- `MinKm()`: Returns the minimum odometer reading and its corresponding date.
- `StatTanken()`: Calculates total liters, total distance covered (based on min/max km from tanken entries), and total cost for fuel-ups.
- `StatKosten()`: Calculates total expenses, proportional expenses (based on depreciation), and tax-relevant proportional expenses.
- `LoadKfzs()`: Loads KFZ data from `kfzs.yaml` using `go-config`.
- `SaveKfzs()`: Saves KFZ data to `kfzs.yaml` using `go-config`.

7. `model/prINTER.go`:

- Handles the formatted output of KFZ statistics to the console.
- Uses `golang.org/x/text/message` for German-localized number formatting.
- `PrintStats()`: The main function to print a comprehensive summary, calling other print functions.
- `printHeader()`: Prints the vehicle name and license plate.
- `printTanken()`: Prints a detailed list of fuel-up entries, including calculated distance, consumption per 100km, and trends.
- `printKosten()`: Prints a detailed list of expense entries, including depreciation information.
- `printSummary()`: Prints a summary of overall statistics: start/current odometer, total distance driven, daily average, fuel-up totals, expense totals (total, proportional, tax-relevant), and overall totals.

Overall Application Flow:

The application starts by loading existing vehicle data. It then presents a menu to the user, allowing them to record new fuel-ups or expenses, export data to Excel, or view a summary. User input is handled by the `input` package, ensuring proper parsing and validation. Data is stored in YAML files. The `model` package manages the data structures and business logic, while the `printer` package is responsible for presenting formatted statistics to the user. The `Makefile` provides convenient commands for building, installing, and managing dependencies.

This project provides a robust and user-friendly CLI tool for managing vehicle-related costs with detailed reporting capabilities.